

# Executive Project Report: Supply Chain Agentic AI using Gurobi Solver

GitHub Repository: <https://github.com/divishajmani/GurobiSupplyChainAgenticAISolver>

## 1. Executive Summary

This report documents the design, implementation, and verification of the **Supply Chain Agentic AI using Gurobi Solver**, a decision intelligence system that converts open-ended, natural language business requests into mathematical optimization models.

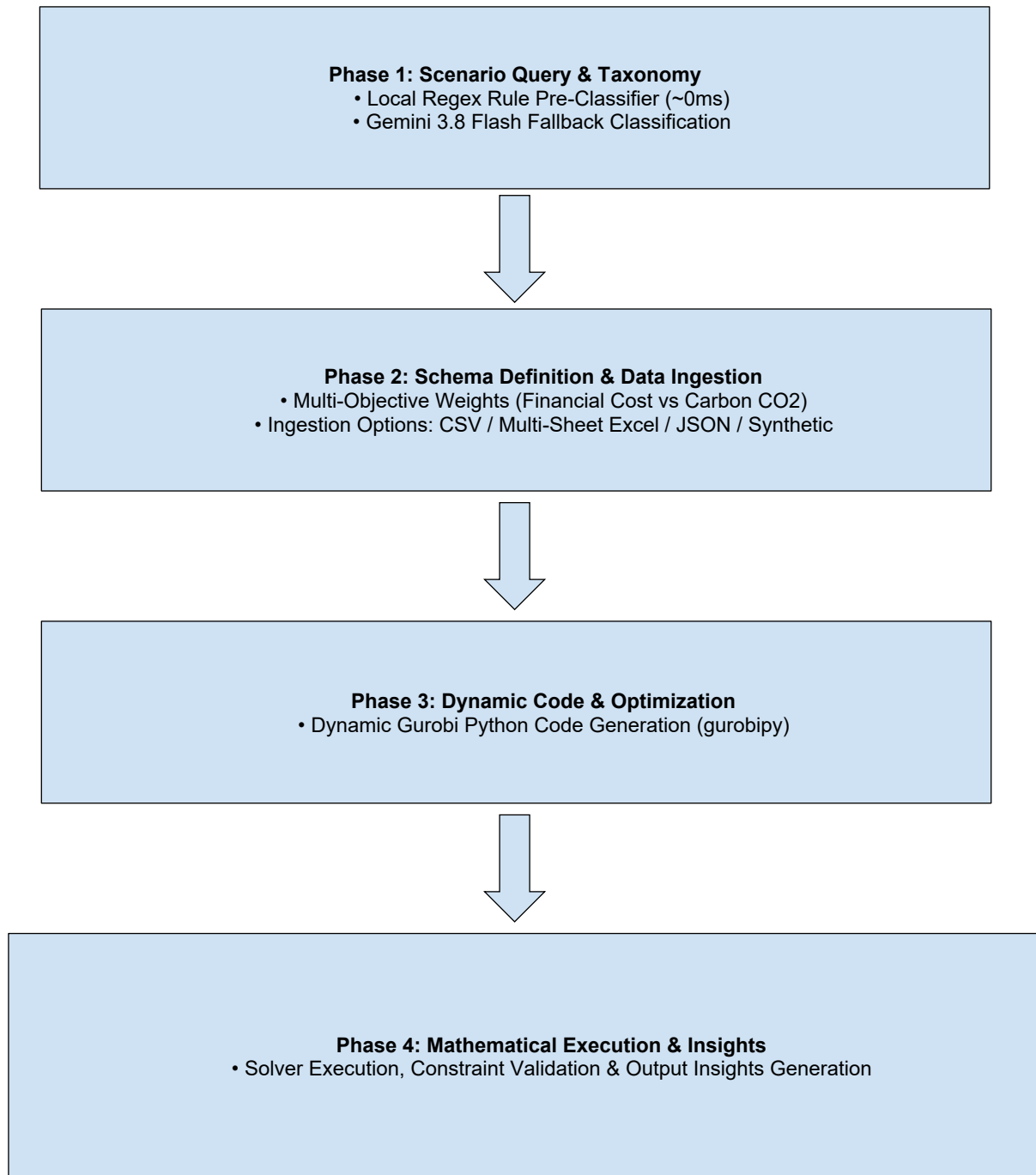
The system combines **multi-agent LLM orchestration** (CrewAI and Google Gemini 3.x), **mixed-integer programming** (Gurobi gurobipy), and a **Streamlit control-tower interface**.

It addresses several challenges associated with deploying traditional operations research (OR) solutions:

- **Natural-Language Mapping:** Translates informal scenarios (for example, “*What if Chicago capacity drops by 50%?*”) into formal operations-research classifications and schemas.
- **Dynamic Code Compilation:** Generates, compiles, and executes gurobipy scripts dynamically based on the incoming data schema.
- **Self-Healing Fallbacks:** Applies penalty-based feasibility relaxation (feasRelaxS) and irreducible inconsistent subsystem analysis (computellS) when data or constraints make a model infeasible.
- **Multimodal Data Ingestion:** Supports CSV files, multisheet Excel workbooks (.xlsx), custom JSON payloads, and LLM-generated benchmark datasets.
- **Visual and Conversational Analytics:** Provides interactive NetworkX visualizations, constraint dual values (shadow prices), executive briefings, and a chatbot with session memory.

## 2. System Architecture & Technical Specifications

The system follows a four-phase sequential workflow:



## Component Stack & Model Assignments

- **User Interface:** Streamlit Dashboard (app.py).
- **Agent Orchestration Framework:** CrewAI (optimizer\_core.py).
- **LLM Engine Assignments:**
  - **llm\_fast (gemini/gemini-3.8-flash):** High-speed classification, schema architecture, tool calling, and Gurobi Python code generation.

- **llm\_cheap (gemini/gemini-3.5-flash-lite)**: Low-cost, fast synthetic benchmark dataset synthesis.
- **llm\_reasoning (gemini/gemini-3.8-flash)**: Multi-step business reasoning, executive report synthesis, and interactive post-solve Q&A.
- **Optimization Engine**: Gurobi Optimizer (gurobipy version 11+).
- **Network Graph Engine**: NetworkX & Matplotlib.

## 3. Core Workflow & Pipeline Stages

### Stage 1: Problem Classification & Taxonomy

The pipeline first evaluates input text with a local regular-expression pre-classifier, which adds negligible latency, to identify common supply-chain problem types such as vehicle routing, inventory, facility location, and production. If no rule matches, it falls back to gemini-3.8-flash.

### Stage 2: Data Ingestion & Deferred Code Compilation

To improve execution safety, gurobipy scripts are compiled only after the data parameters are finalized. Multi-objective priority sliders let users dynamically balance financial cost minimization against carbon-footprint minimization.

### Stage 3: Execution & Self-Healing Optimization

The DynamicGurobiExecutionTool runs generated code in an isolated `exec()` context. When constraints conflict, the self-healing handler identifies IIS boundaries and applies penalty slack variables so that the application can return a `RELAXED_OPTIMAL` solution instead of terminating the session.

### Stage 4: Executive Briefing & Contextual Chatbot

The Executive Logistics Strategist agent translates mathematical outputs and constraint shadow prices into clear business insights. A session-aware chatbot provides step-by-step guidance and generates tailored Markdown prompts when a follow-up request requires changes to the optimization model.

## 4. How to Run the Application

Follow these steps to install and launch the Agentic Supply Chain Decision Control Tower locally. Run each command in a terminal or command prompt from the repository root unless otherwise noted.

## Prerequisites

- Python 3.10 or later.
- Git.
- A valid Gemini API key.
- A working Gurobi installation with a valid license for your environment.

## Step 1: Clone the Repository

Clone the repository and switch to the project directory:

```
git clone https://github.com/divishajmani/GurobiSupplyChainAgenticAISolver.git
cd GurobiSupplyChainAgenticAISolver
```

## Step 2: Create and Activate a Virtual Environment

Create an isolated Python environment, then activate it using the commands for your operating system:

macOS or Linux

```
python3 -m venv venv
source venv/bin/activate
```

Windows Command Prompt

```
python -m venv venv
venv\Scripts\activate
```

*After activation, your terminal prompt should display the environment name.*

## Step 3: Install the Required Dependencies

Install the pinned project dependencies:

```
python -m pip install --upgrade pip
pip install -r requirements.txt
```

*Using the requirements file helps keep package versions consistent across environments.*

## Step 4: Configure Credentials

Create a `.env` file in the project root and add these values:

```
GEMINI_API_KEY=your_actual_gemini_api_key_here
OTEL_SDK_DISABLED=true
CREWAI_TELEMETRY_OPT_OUT=true
```

**Important:** Never commit the `.env` file or expose API keys in source control.

If your Gurobi installation uses a license file, also set:

```
GRB_LICENSE_FILE=/path/to/gurobi.lic
```

## Step 5: Launch the Control Tower

Start the Streamlit interface from the project root:

```
streamlit run app.py
```

If the browser does not open automatically, go to `http://localhost:8501`. Keep the terminal open while using the application, and press `Ctrl+C` to stop the server.

## Step 6: Verify the Installation

- Confirm that the Streamlit page loads without import or credential errors.
- Submit a small sample scenario and verify that the workflow reaches the optimization and executive-briefing stages.
- If startup fails, confirm that the virtual environment is active, dependencies are installed, and the `.env` file is located in the repository root.

# 5. Verification & Testing Summary

The system is validated with an automated suite of 20 tests in `test_optimizer.py`:

| Category                             | Test Case Range | Coverage Areas                                                                                                         |
|--------------------------------------|-----------------|------------------------------------------------------------------------------------------------------------------------|
| <b>Taxonomy &amp; Classification</b> | TC-01–TC-06     | Regular-expression matching, LLM fallback behavior, vehicle routing, facility location, and production classification. |

|                         |             |                                                                                                                           |
|-------------------------|-------------|---------------------------------------------------------------------------------------------------------------------------|
| Schema & Data Ingestion | TC-07–TC-10 | Schema generation, synthetic JSON creation, malformed-string repair, and zero-demand boundary handling.                   |
| Solver & Code Execution | TC-11–TC-15 | <code>run_model(data)</code> compilation, infeasibility recovery, multi-objective weighting, and shadow-price extraction. |
| Conversational Chatbot  | TC-16–TC-20 | Session-memory retention, rerun-prompt generation, tool-error recovery, and report generation.                            |

## How to Test the Code

Use the following steps to run the automated test suite locally:

1. Confirm that the virtual environment is active and that a valid API key is defined in the `.env` file.
2. Run the standard test suite:

```
pytest test_optimizer.py
```

3. For verbose logs and captured output, run:

```
pytest test_optimizer.py -v -s
```

## 6. Future Work & Enhancements

While the Control Tower successfully bridges natural language input with deterministic integer programming, future iterations can expand its enterprise capabilities:

- **Full-Stack Enterprise Frontend Migration:** Transition the user interface from Streamlit to a React/Next.js single-page application paired with a Node.js/Express backend service to support asynchronous job queues, WebSocket live updates, and interactive Cytoscape.js network visualizers.
- **Automated Adversarial Risk Stress-Testing:** Introduce a dedicated **Adversarial Stress-Test Agent** that automatically inspects optimal baseline decisions, identifies top single points of failure (e.g., maximum node throughput), and runs automated counterfactual disruption scenarios.
- **Multi-Objective Pareto Frontier Curves:** Expand the multi-objective optimization handler beyond static priority weights to compute full 2D Pareto optimal trade-off curves (Financial Cost vs.  $CO_2$  Footprint vs. Lead Time).
- **Real-Time Data Integration:** Extend data ingestion beyond static CSV/Excel files to include live enterprise integrations via REST APIs and database connectors (e.g., SAP, Oracle SCM, PostgreSQL).

- **Stochastic & Chance-Constrained Programming:** Upgrade Gurobi formulations from deterministic MILPs to stochastic optimization models to incorporate dynamic demand variance, lead time uncertainty, and probabilistic supply failures.